

Lecture 22: Touch the Network

Scapy, Mininet, and Sockets

EC 441 — Introduction to Computer Networking
Boston University, Spring 2026

Tuesday, April 21, 2026

Learning Objectives

By the end of this lecture you should be able to:

1. Craft, send, and sniff packets in Python using Scapy, and map every field of a crafted packet back to the protocol concepts from L13–L20.
2. Build a small emulated topology in Mininet, run traffic between hosts, and vary link parameters (bandwidth, delay, loss) to shape the behavior of a real TCP connection.
3. Write a minimal TCP and UDP client/server using the Python `socket` module, and explain the asymmetries between the two transport-layer APIs.
4. Close the full read + write loop: write an app with sockets, capture its traffic with `tcpdump` or `pyshark`, and inspect packets with Scapy.

Framing

L21 was about *reading* the network — watching the traffic already there. L22 is about *writing* to it. Three tools, one arc:

1. **Scapy** — craft a single packet, byte by byte.
2. **Mininet** — own a whole topology.
3. **Sockets** — the interface real applications use.

At each stage you are an active participant, not an observer. By the end of Session 2 you can both generate traffic and inspect it — the full read + write loop.

All demos run inside the `ec441` Multipass VM. Raw-socket operations (Scapy) and network-namespace operations (Mininet) require root, so `sudo` appears throughout.

1 Scapy: Crafting Packets

Scapy is a Python library for building, sending, receiving, and dissecting packets at any layer. It is *Wireshark in reverse*: instead of parsing bytes off the wire into Python objects, you build Python objects and Scapy writes the bytes onto the wire.

Scapy opens raw sockets (`AF_PACKET` / `SOCK_RAW`) to bypass the kernel's normal socket layer, so every send and every sniff requires root privileges.

```
sudo pip install scapy
sudo python3
```

1.1 Building a packet

The core Scapy idiom is layer-stacking with the `/` operator. Read it as “encapsulated in.”

Build and inspect a packet

Script: `demo_scapy_build_l22.py`

```
from scapy.all import IP, ICMP

pkt = IP(dst="8.8.8.8") / ICMP()
pkt.show()           # pretty-print every field with defaults
bytes(pkt)           # the actual wire bytes
```

Scapy fills in sensible defaults for every field you did not specify: `version=4`, `ihl=5`, `ttl=64`, `proto=1` (ICMP). Every one of these was on the board in L14 or L17 — this is the first moment you can touch them.

1.2 Ping from scratch

`sr1` means “send and receive one.” It sends a packet and returns the first matching reply.

Craft and send an ICMP Echo Request

Script: `demo_scapy_icmp_l22.py`

```
from scapy.all import IP, ICMP, sr1

req = IP(dst="8.8.8.8") / ICMP()
reply = sr1(req, timeout=2, verbose=0)
reply.show()
```

The reply's TTL tells you how many hops it took to come back — count down from 64 (Linux default) or 128 (Windows default) to estimate the hop count. This matches what `ping -c 1` would report; Scapy just gives you full control over what is sent.

1.3 TCP SYN probe

A SYN packet with no listener on the other side triggers a RST (L18). With a listener, the reply is SYN-ACK. This is the core of every port scanner.

SYN probe against an open and a closed port

Script: `demo_scapy_syn_l22.py`

```
from scapy.all import IP, TCP, sr1, RandShort

# Closed port -- expect RST
r1 = sr1(IP(dst="scanme.nmap.org")
        / TCP(dport=81, flags="S", sport=RandShort()),
        timeout=2, verbose=0)
print(r1.sprintf("%TCP.flags%")) # 'R' or 'RA'

# Open port -- expect SYN-ACK
r2 = sr1(IP(dst="scanme.nmap.org")
        / TCP(dport=80, flags="S", sport=RandShort()),
        timeout=2, verbose=0)
print(r2.sprintf("%TCP.flags%")) # 'SA'
```

Ethics

Scanning hosts you do not own may violate the US Computer Fraud and Abuse Act (CFAA) and analogous laws in other jurisdictions. `scanme.nmap.org` is maintained by the nmap project specifically as a legal target for learners; your own VMs are also fine. *Any other target requires written permission.* Network measurement tools double as attack tools — the difference is authorization.

1.4 Traceroute in ten lines

Traceroute is not a protocol — it is a side effect of how IP forwarding decrements TTL (L13) and how ICMP Time Exceeded (L17) works. Any language with raw sockets can implement it in a short loop.

Minimal traceroute

Script: `demo_scapy_traceroute_l22.py`

```
from scapy.all import IP, ICMP, sr1

for ttl in range(1, 16):
    reply = sr1(IP(dst="8.8.8.8", ttl=ttl) / ICMP(),
                timeout=2, verbose=0)
    if reply is None:
        print(f"{ttl:2d}  *")
    elif reply.type == 0: # Echo Reply = reached
        print(f"{ttl:2d} {reply.src} (reached)")
        break
    else: # Time Exceeded = hop
        print(f"{ttl:2d} {reply.src}")
```

1.5 Sniff mode

Scapy also captures. Same libpcap/BPF filter syntax as `tcpdump` and `pyshark` from L21 — different surface on the same machinery.

```
from scapy.all import sniff

pkts = sniff(filter="icmp", count=5, iface="any")
pkts.summary()
```

When do you reach for which tool?

- `tcpdump` — quick one-off CLI capture.
- `pyshark` — bulk read-only analysis of a capture.
- `Scapy` — whenever you want to *both* send and capture in the same script.

2 Mininet: Emulating a Topology

Mininet creates virtual hosts using **Linux network namespaces** and connects them with virtual Ethernet pairs (`veth`). Each “host” is a real Linux process with its own network stack. This is not simulation — it is a real network, in miniature, on a single kernel.

With ten lines of Python you can stand up a topology with controllable bandwidth, delay, and loss. This is how protocol changes get tested before shipping to physical routers.

```
sudo apt-get install mininet
# or, if the distro version is old:
sudo pip install mininet
```

2.1 Hello world

Three hosts on one switch

```
sudo mn --topo single,3 --link tc

mininet> h1 ping -c 3 h2
mininet> pingall
mininet> exit
```

`--link tc` tells Mininet to use the Linux traffic-control subsystem (`tc`) for link emulation, which is what makes bandwidth and delay parameters work in later demos.

2.2 Bottleneck topology

The interesting case: two hosts with an asymmetric link that governs the whole flow.

Bottleneck topology with iperf3

Script: `demo_mininet_bottleneck_l22.py`. Run with `sudo python3`.

```
from mininet.net import Mininet
from mininet.link import TCLink
from mininet.node import OVSController

net = Mininet(controller=OVSController, link=TCLink)
net.addController('c0')

h1 = net.addHost('h1')
h2 = net.addHost('h2')
s1 = net.addSwitch('s1')

net.addLink(h1, s1, bw=100, delay='1ms')
net.addLink(s1, h2, bw=10, delay='20ms', loss=1)

net.start()
h2.cmd('iperf3 -s -D')
print(h1.cmd('iperf3 -c %s -t 20' % h2.IP()))
net.stop()
```

What to watch for while it runs:

- **Bandwidth** settles near 10 Mb/s — the bottleneck governs.
- **Loss** of 1% triggers TCP fast retransmit and multiplicative decrease (L20); throughput drops noticeably below a loss-free 10 Mb/s.
- **Delay** of 20 ms one-way gives a 40 ms RTT.
- **BDP** = 10 Mb/s $\times$ 40 ms $\approx$ 50 KB $\approx$ 35 MSS — cwnd has room to grow.

Why Mininet succeeds where L21 loopback failed

In L21 the iperf demo ran over loopback. Loopback has no loss and sub-millisecond RTT, so cwnd stayed at the initial window (`iw=10`) forever — the AIMD arc was invisible. Mininet adds real delay and real loss. *This* is where `ss -tipm` shows meaningful cwnd dynamics: slow start, ssthresh, multiplicative decrease on loss, recovery.

Inside any Mininet host you can run every CLI tool from L21 in its private namespace. In the same Python script:

```
# during iperf, same script:
print(h1.cmd('ss -tipm'))
```

Now `ss` reports the congestion window for a connection whose RTT and loss you control.

3 Socket Programming

Sockets are the interface between application code and every layer below. Every networked program you have ever used — browsers, SSH, `dig`, `curl` — sits on top of this API.

3.1 The API at a glance

TCP server flow:

`socket() → bind() → listen() → accept() → recv/send → close()`

TCP client flow:

`socket() → connect() → send/recv → close()`

UDP (symmetric):

`socket() → bind() → sendto/recvfrom → close()`

The key differences before any code:

- UDP has no `connect`, `listen`, or `accept` — there is no connection to set up.
- UDP's `sendto/recvfrom` always take an address; TCP's `send/recv` do not because the socket is already bound to a peer.
- TCP gives you a stream of bytes; UDP gives you discrete datagrams.

3.2 TCP echo server and client

Minimal TCP echo pair

Scripts: `demo_tcp_echo_server_l22.py` and `demo_tcp_echo_client_l22.py`.

Server:

```
import socket
HOST, PORT = '127.0.0.1', 5001

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((HOST, PORT))
srv.listen(1)

while True:
    conn, addr = srv.accept()
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)
```

Client:

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(('127.0.0.1', 5001))
    s.sendall(b'hello from ec441\n')
    print(s.recv(1024))
```

Three traps worth naming now, before they bite in the project:

Three socket traps

1. **SO_REUSEADDR.** Without it, restarting the server within roughly 60 seconds fails with “*Address already in use.*” The port is stuck in `TIME_WAIT` (L19). Every server sets this option.
2. **recv(N) is a hint.** TCP is a byte stream, not a message-preserving channel. A call to `recv(1024)` may return anywhere from 1 to 1024 bytes. Applications must do their own framing. HTTP uses `Content-Length` and `\r\n\r\n` delimiters; length-prefix framing is another common pattern.
3. **sendall vs send.** `send` may write fewer bytes than requested when kernel buffers are full. `sendall` loops until everything is written. Prefer `sendall` in almost all cases.

3.3 UDP echo server and client

Minimal UDP echo pair

Scripts: `demo_udp_echo_server_l22.py` and `demo_udp_echo_client_l22.py`.

Server:

```
import socket

srv = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
srv.bind(('127.0.0.1', 5002))

while True:
    data, addr = srv.recvfrom(1024)
    srv.sendto(data, addr)
```

Client:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b'hello udp\n', ('127.0.0.1', 5002))
data, addr = s.recvfrom(1024)
print(data, 'from', addr)
```

What is missing from the UDP server, compared with TCP: no `listen`, no `accept`, no per-connection socket. UDP is stateless at the socket layer — every datagram stands alone. A practical consequence is that if the client runs before the server starts, the first datagram is simply lost without an error.

Aspect	TCP	UDP
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection setup	listen/accept/connect	none
Message boundaries	byte stream (framed by app)	preserved per datagram
Retransmission	kernel	application's job
Ordering	guaranteed	none
Typical users	HTTP, SSH, Git, SMTP	DNS, QUIC, video, games

3.4 setsockopt options worth knowing

Option	Effect	Set when
SO_REUSEADDR	Immediate rebind after close	always on servers
TCP_NODELAY	Disable Nagle's algorithm	interactive / RPC
SO_KEEPALIVE	Periodic probes on idle TCP	long-lived connections
SO_RCVBUF / SNDBUF	Socket buffer size	high-BDP paths

Nagle's algorithm

Nagle batches small writes into larger TCP segments to reduce the per-packet overhead of headers. It is a small efficiency win for bulk transfer and a latency disaster for interactive traffic (SSH, RPC). Default is *on*; you disable it per-socket with `TCP_NODELAY`. This is the single most-cited `setsockopt` option in real-world application code.

4 Close the Loop

Run all four pieces at once inside the `ec441` VM:

```
# shell 1 -- capture
sudo tcpdump -i lo -w /tmp/sockets.pcap \
    'port 5001 or port 5002'

# shell 2 -- servers
python3 demo_tcp_echo_server_l22.py &
python3 demo_udp_echo_server_l22.py &

# shell 3 -- clients
python3 demo_tcp_echo_client_l22.py
python3 demo_udp_echo_client_l22.py

# shell 1 -- stop capture
sudo pkill tcpdump
```

Now analyze the capture with the L21 tooling:

```
import pyshark
cap = pyshark.FileCapture('/tmp/sockets.pcap')
for p in cap:
    print(p.highest_layer, p.length, p.transport_layer)
```

Today's payoff

The `sockets.pcap` you just captured contains:

- An **Ethernet frame** (L08) carrying
- an **IP datagram** (L14–L17) carrying
- a **TCP segment with sequence numbers, ACKs, and a congestion window** (L18–L20) carrying
- application bytes generated by *your own Python code*.

You wrote the application. You emulated the network path. You captured the traffic. You can explain every field. The semester — as a protocol stack — is in this one pcap.

References

- Wikipedia: Berkeley sockets — primary student reference for the socket API.
- Wikipedia: Nagle's algorithm.
- Wikipedia: Linux namespaces — the kernel feature Mininet rides on.
- Scapy documentation: <https://scapy.readthedocs.io/>
- Mininet walkthrough: <http://mininet.org/walkthrough/>
- Mininet Python API: <http://mininet.org/api/>
- Python socket module: <https://docs.python.org/3/library/socket.html>
- Python socket HOWTO: <https://docs.python.org/3/howto/sockets.html>
- `man 7 socket`, `man 7 tcp`, `man 7 udp` — authoritative.